LAB SHEET 7

Experiment Name: Motion Estimation using Optical Flow Algorithms in Video Sequences

Steps :

1. Import the required Python libraries and load a video sequence or access a live webcam stream using OpenCV.
2. Read consecutive video frames and convert them to grayscale for optical flow computation.
3. Implement the Lucas-Kanade Optical Flow algorithm to estimate the motion of selected feature points across consecutive frames.
4. Visualize the motion vectors by drawing arrows or trajectories representing the movement of tracked points.
5. Implement the Farneback Dense Optical Flow algorithm to estimate motion for every pixel in the image.
6. Display the dense optical flow output using color-coded motion visualization to represent direction and magnitude.
7. Compare the performance of Lucas-Kanade and Farneback algorithms in terms of accuracy, computational complexity, and robustness.
8. Analyze the effect of object speed, lighting conditions, and camera motion on optical flow estimation.
9. Evaluate the suitability of each algorithm for different real-world applications such as object tracking, surveillance, and autonomous navigation.
10. Summarize the observations and discuss the significance of optical flow in dynamic scene analysis and motion-based computer vision systems.

Solution -> opticalflow.py

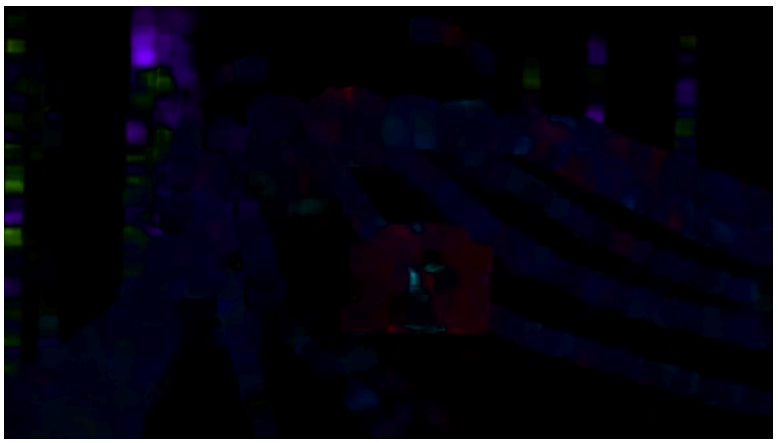
```
import cv2
import numpy as np
import os
INPUT_VIDEO = "input.mp4"
os.makedirs("outputs/lucas_kanade", exist_ok=True)
os.makedirs("outputs/farneback", exist_ok=True)
video = cv2.VideoCapture(INPUT_VIDEO)
if not video.isOpened():
    print("Error: Cannot open input.mp4")
    exit()
ret, old_frame = video.read()
if not ret:
    print("Error: Cannot read first frame.")
    video.release()
    exit()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
```

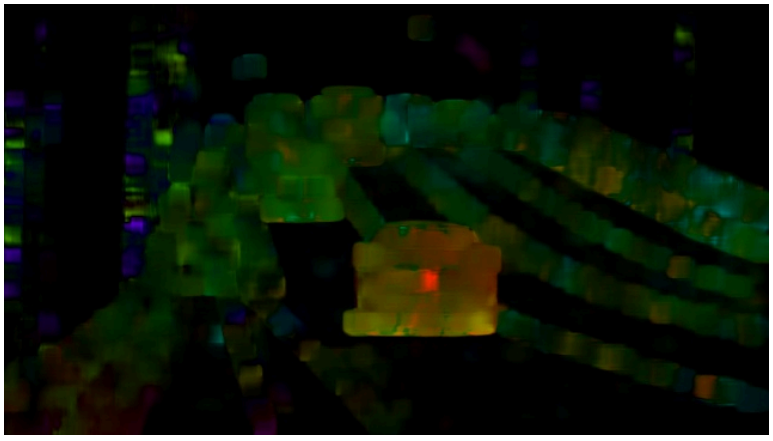
```
    minDistance=7,
    blockSize=7
)
old_points = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
mask = np.zeros_like(old_frame)
frame_count = 0
while True:
    ret, frame = video.read()
    if not ret:
        break
    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )
    new_points, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        old_points,
        None,
        **lk_params
    )
    if new_points is not None:
        good_new = new_points[status == 1]
        good_old = old_points[status == 1]
        for new, old in zip(good_new, good_old):
            x_new, y_new = new.ravel()
            x_old, y_old = old.ravel()
            mask = cv2.line(
                mask,
                (int(x_old), int(y_old)),
                (int(x_new), int(y_new)),
                (255, 255, 255),
                2
            )
```

```
            frame = cv2.circle(
                frame,
                (int(x_new), int(y_new)),
                4,
                (0, 255, 0),
                -1
            )
        output = cv2.add(frame, mask)
        if frame_count % 10 == 0:
            cv2.imwrite(
                f"outputs/lucas_kanade/"
                f"frame_{frame_count}.jpg",
                output
            )
        old_gray = frame_gray.copy()
        old_points = good_new.reshape(
            -1,
            1,
            2
        )
    frame_count += 1
video.release()
print("Lucas-Kanade completed.")
video = cv2.VideoCapture(INPUT_VIDEO)
ret, old_frame = video.read()
if not ret:
    print("Error: Cannot read video.")
    video.release()
    exit()
old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)
frame_count = 0
while True:
    ret, frame = video.read()
    if not ret:
        break
    new_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        new_gray,
        None,
        0.5,
        3,
```

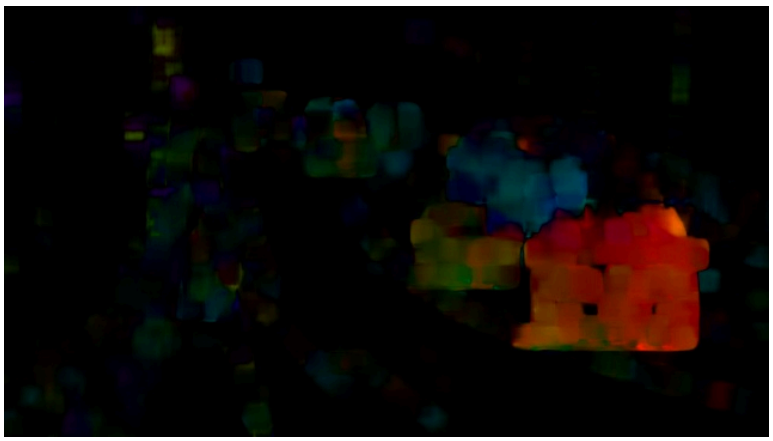
```
            15,
            3,
            5,
            1.2,
            0
        )
        magnitude, angle = cv2.cartToPolar(
            flow[..., 0],
            flow[..., 1]
        )
        hsv = np.zeros_like(frame)
        hsv[..., 0] = (
            angle * 180 / np.pi / 2
        )
        hsv[..., 1] = 255
        hsv[..., 2] = cv2.normalize(
            magnitude,
            None,
            0,
            255,
            cv2.NORM_MINMAX
        )
        optical_flow = cv2.cvtColor(
            hsv,
            cv2.COLOR_HSV2BGR
        )
        if frame_count % 10 == 0:
            cv2.imwrite(
                f"outputs/farneback/"
                f"frame_{frame_count}.jpg",
                optical_flow
            )
        old_gray = new_gray
        frame_count += 1
video.release()
print("Farneback Dense Optical Flow completed.")
print("Experiment 7 completed successfully.")
```

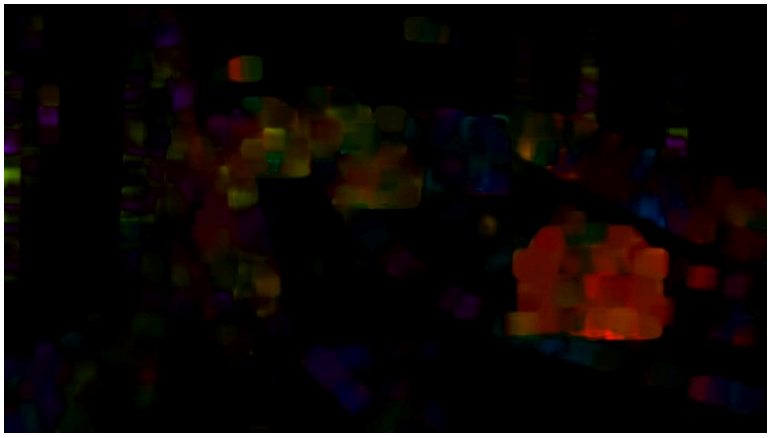
Output -> farneback:











lucas_kanade:

